

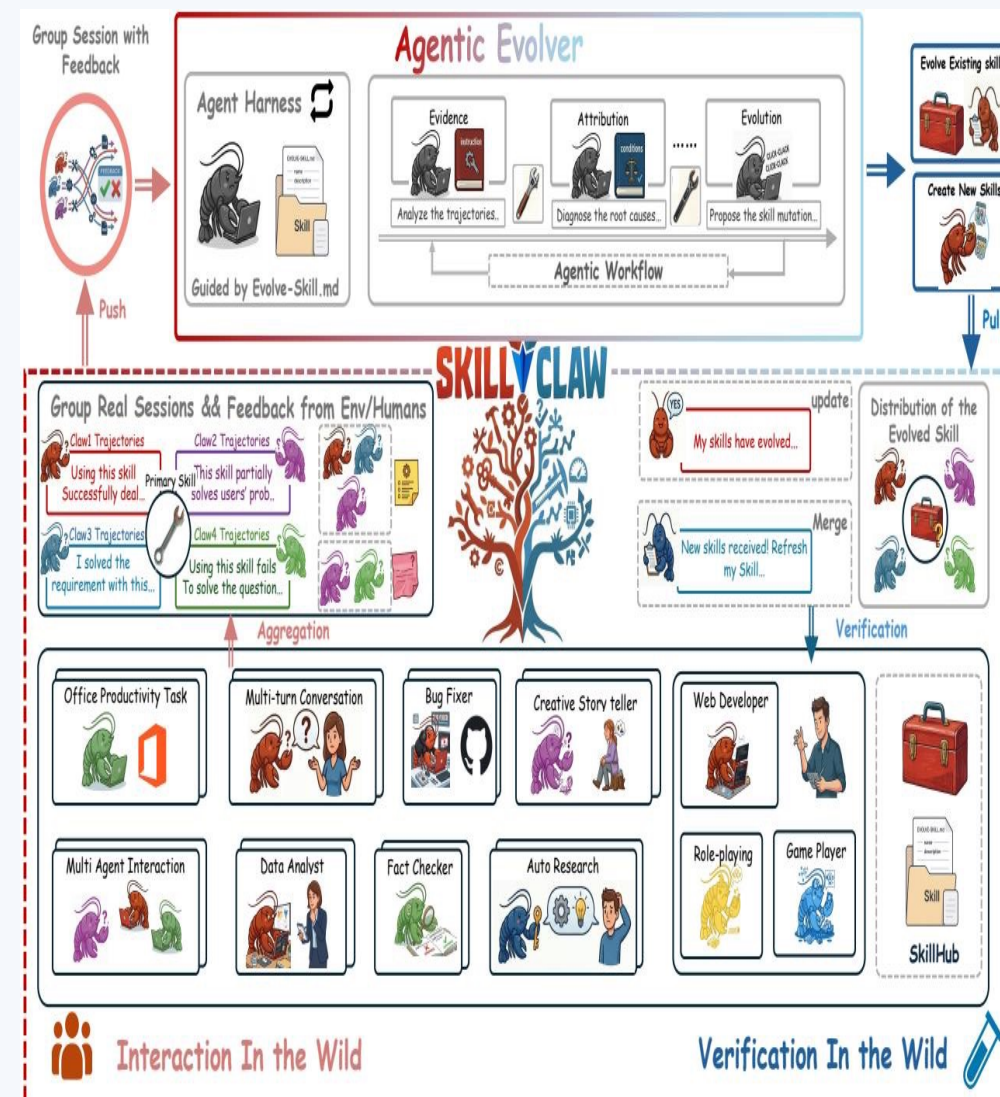
SkillClaw: Collective Skill Evolution for Multi-User LLM Agent Ecosystems

Let Skills Evolve Collectively with Agentic Evolver

Cross-user trajectories become shared skill intelligence

Closed-loop evolution: Evidence → Attribution → Evolution → Distribution

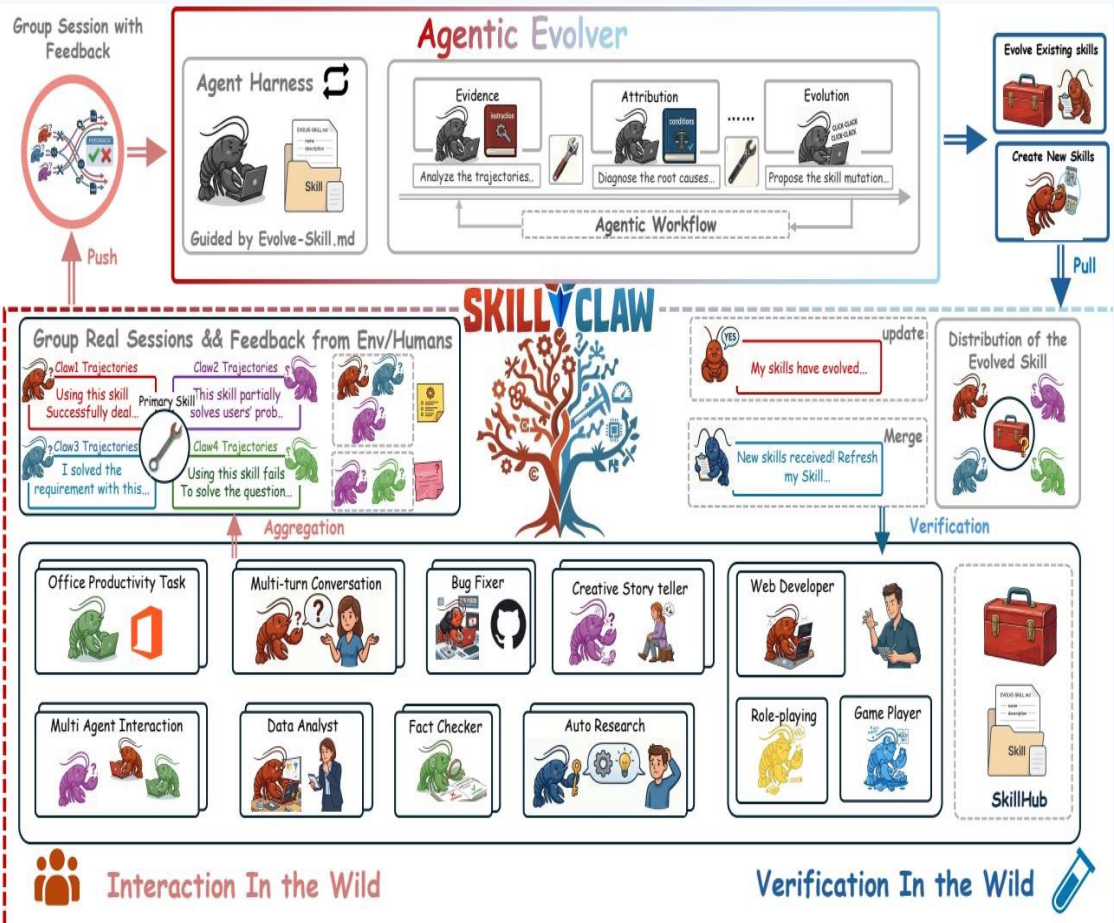
Source: arXiv:2604.08377



Static Skills Create Repeated Rediscovery

Today: skills are manually installed and remain static after deployment
Users repeatedly hit similar failures and independently rediscover fixes
No system-level accumulation of runtime knowledge across users
Result: slower reliability gains and duplicated troubleshooting effort

Closed-Loop Architecture Overview



Multi-user interactions produce diverse trajectories around shared skills
Evidence stage aggregates sessions and extracts behavioral signals
Attribution stage diagnoses why a skill succeeds or fails
Evolution stage proposes skill mutations; Distribution syncs validated updates

Inside the Agentic Evolver

1) Evidence

2) Attribution

3) Evolution

Evidence: analyze trajectories for recurring success/failure patterns

Attribution: infer root causes and boundary conditions of skill behavior

Evolution: generate candidate skill mutations via open-ended reasoning

Validator gate accepts only non-regressive updates before release

Why Cross-User Aggregation Matters

Compatible systems: OpenClaw, CoPaw, IronClaw, PicoClaw, ZeroClaw, NanoClaw, NemoClaw

Different users exercise the same skill under diverse contexts and constraints

Aggregated signals reveal generalizable boundaries, not one-off fixes

Single-user data is often insufficient to separate robust improvements from idiosyncrasies

Case Study A: Multimodal Creative Pipeline

Date	Outcome	Rationale
Day 1	Rejected	Output style drift; failed consistency checks
Day 2	Rejected	Improved speed but reduced multimodal coherence
Day 3	Accepted	Better tool ordering and stable quality across prompts
Day 4	Rejected	Regression on long-context creative tasks
Day 5	Rejected	Overfit to narrow prompt pattern
Day 6	Rejected	No net gain vs. current skill baseline

Case Study B: Git Push Auth-Fallback

Date	Outcome	Rationale
Day 1	Accepted	Added reliable auth fallback path for common failure mode
Day 2	Rejected	Increased retries caused unnecessary latency
Day 3	Accepted	Refined credential detection and reduced false triggers
Day 4	Rejected	Edge-case regression in non-interactive environments
Day 5	Accepted	Stabilized fallback ordering and error messaging
Day 6	Rejected	No additional improvement over Day 5 variant

Three Distinguishing Properties & Implications

Collective evolution: individual interactions continuously improve a shared ecosystem

Fully automatic: runtime-driven evolution without manual curation

Agentic paradigm: open-ended reasoning replaces fixed update rules

Implication: move from static agent deployment to continuously improving systems